IS1201 - Programming and Problem Solving

Understanding and Using Pointers in C

Pointers and Memory

When a C program is compiled, it works with three types of memory:

1. **Static/Global**
   - Statically declared variables and global variables are allocated to this type of memory
   - They are allocated when the program starts and remain in existence until the program terminates
   - While all functions have access to global variables, the scope of static variables is restricted to their defining function

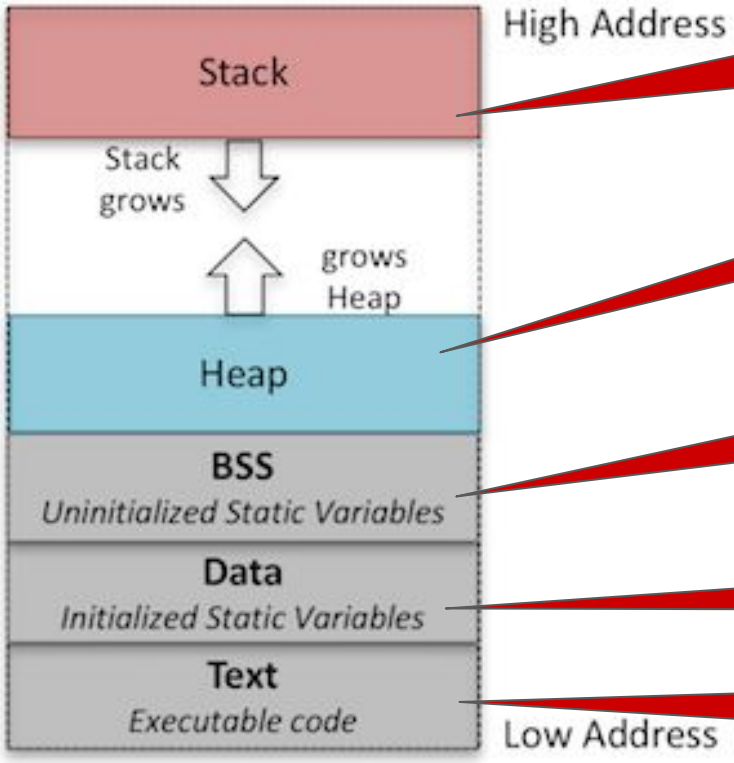
2. **Automatic**
   - These variables are declared within a function and are created when a function is called
   - Their scope is restricted to the function, and their lifetime is limited to the time the function is executing

Pointers and Memory

3. **Dynamic**
   - Memory is allocated from the heap and can be released as necessary
   - A pointer references the allocated memory
   - The scope is limited to the pointer or pointers that reference the memory
   - Exists until memory is released

	Scope	Lifetime
Global	The entire file	The lifetime of the application
Static	The function it is declared within	The lifetime of the application
Automatic (local)	The function it is declared within	While the function is executing
Dynamic	Determined by the pointers that reference this memory	Until the memory is freed

Pointers and Memory



Stores local variables, function parameters, and return addresses. Managed automatically. Grows downward. Example: variables declared inside a function live on the stack

Used for dynamic memory allocation Managed manually by the programmer. Grows upward

Uninitialized Data (Block Started by Symbol): global/static variables without values.
Example: static int y; // default 0

Global/Static variables with initial values.
Example: int x = 5;

Stores compiled machine code (your program's instructions). Usually read-only to prevent accidental modification

Uses of Pointers

- Creating fast and efficient code
- Providing a convenient means for addressing many types of problems
- Supporting dynamic memory allocation
- Making expressions compact and succinct
- Providing the ability to pass data structures by pointer without incurring a large overhead
- Protecting data passed as a parameter to a function

What is a Pointer?

- A **pointer** is a **variable** that **stores** the **memory address** of **another variable**
- Instead of holding a direct value (like 10 or 3.14), it holds the location in memory where that value is stored

Declaring Pointers:

- Pointer variables are declared using a data type followed by an asterisk and then the pointer variable's name
- In the following example, an integer and a pointer to an integer are declared:

```
int num;
int *pi;
```

- The use of white space around the * is a matter of user preference

Address Operator

- The address of operator, **&**, will return its operand's address
- We can initialize the pi pointer with the address of num using this operator as follows:

```
int num = 0;
int *pi = &num;
```

Displaying Pointer Values:

```
int num = 0;
int *pi = &num;

printf("Memory address of variable num: %d, Value of variable num: %d \n", &num, num);
printf("Memory address of variable pi: %d, Value of variable pi: %d \n", &pi, pi);
```

When executed, you may get output as follows:

```
Memory address of variable num: 6291148, Value of variable num: 0
Memory address of variable pi: 6291136, Value of variable pi: 6291148
```

Field Specifiers

- The printf function has a couple of other field specifiers useful when displaying pointer values:

Specifier	Meaning
%x	Displays the value as a hexadecimal number.
%o	Displays the value as an octal number.
%p	Displays the value in an implementation-specific manner; typically as a hexadecimal number.

Example:

```
int num = 0;
int *pi = &num;

printf("Memory address of variable pi: %d, Value of variable pi: %d \n", &pi, pi);
printf("Memory address of variable pi: %x, Value of variable pi: %x \n", &pi, pi);
printf("Memory address of variable pi: %o, Value of variable pi: %o \n", &pi, pi);
printf("Memory address of variable pi: %p, Value of variable pi: %p \n", &pi, pi);
```

```
Memory address of variable pi: 6291136, Value of variable pi: 6291148
Memory address of variable pi: 5ffec0, Value of variable pi: 5ffecc
Memory address of variable pi: 27777300, Value of variable pi: 27777314
Memory address of variable pi: 00000000005FFEC0, Value of variable pi: 00000000005FFECC
```

Dereferencing a Pointer Using the Indirection Operator

- The indirection operator, *****, returns the value pointed to by a pointer variable
- In the following example, num and pi are declared and initialized:

```
int num = 5;
int *pi = &num;
```

- The indirection operator is then used in the following statement to display 5, the value of num:

```
printf("%p\n",*pi); // Displays 5
```

- We can also use the result of the dereference operator as an lvalue
- The term ***lvalue*** refers to the **operand found on the left side of the assignment operator**
- All lvalues must be modifiable since they are being assigned a value

Dereferencing a Pointer Using the Indirection Operator

- The following will assign 200 to the integer pointed to by pi. Since it is pointing to the variable num, 200 will be assigned to num

```
int num = 5;
int *pi = &num;

// Assign 200 to the memory location pointed by 'pi'
*pi = 200;

printf("%d\n",num); // Displays 200
```

NULL in Pointers

- When **NULL** is assigned to a pointer, it means the pointer does not point to anything
- It does not point to any area of memory
- Two null pointers will always be equal to each other
- The use of **NULL** and **0** are language-level symbols that represent a null pointer
- So we can assign a zero to a pointer, but we cannot assign any other integer value

- A null pointer and an uninitialized pointer are different. An uninitialized pointer can contain any value, whereas a pointer containing NULL does not reference any location in memory

```
int num = 5;
int *pi = &num;

pi = 0;
pi = NULL;
pi = 100; // Syntax error
pi = num; // Syntax error
```

NULL in Pointers

- A pointer can be used as the sole operand of a logical expression
- For example, we can test to see whether the pointer is set to NULL using the following sequence:

```
if(pi){
    // Not NULL
}else{
    // Is NULL
}
```

Pointer to void

- A pointer to void is a general-purpose pointer used to hold references to any data type
- An example of a pointer to void is shown below:

```
void *pv;
```

- It has two interesting properties:
  - A pointer to void will have the same representation and memory alignment as a pointer to char
  - A pointer to void will never be equal to another pointer. However, two void pointers assigned a NULL value will be equal

Pointers to void

- Any pointer can be assigned to a pointer to void. It can then be cast back to its original pointer type. When this happens the value will be equal to the original pointer value

```
int num;
int *pi = &num;
printf("Value of pi: %p\n", pi);
void *pv = pi;
pi = (int *) pv;
printf("Value of pi: %p\n", pi);
```

- When this sequence is executed as shown below, the pointer address is the same:

```
Value of pi: 00000000005FFEBC
Value of pi: 00000000005FFEBC
```

Global and Static pointers

- If a pointer is declared as global or static, it is initialized to NULL when the program starts
- An example of a global and static pointer follows:

```
#include <stdio.h>

int *globalpi; //global pointer

void foo(){
    static int* ststicpi; // static pointer
    // .......
}

int main(){
    // ......
    return 0;
}
```

Pointer Size and Type

- On most modern platforms, the size of a pointer to data is normally the same regardless of the pointer type
- A pointer to a char has the same size as a pointer to a structure or int
- C standard does not dictate that size be the same for all data types
- However, the size of a pointer to a function may be different from the size of a pointer to data
- The size of a pointer depends on the machine in use and the compiler
- For example, on modern versions of Windows the pointer is 32 or 64 bits in length

```c
#include <stdio.h>

int main(){
    int *pi;     // pointer to int
    char *pc;    // pointer to char
    float *pf;   // pointer to float

    // Print the size of each pointer type
    printf("Size of the pointer to an integer : %zu\n", sizeof(pi));
    printf("Size of the pointer to a char     : %zu\n", sizeof(pc));
    printf("Size of the pointer to a float    : %zu\n", sizeof(pf));

    return 0;
}
```

```
Size of the pointer to an integer : 8
Size of the pointer to a char     : 8
Size of the pointer to a float    : 8
```

Pointer Operators

- There are several operators available for use with pointers other than dereference and address of operators

Operator	Name	Meaning
*		Used to declare a pointer
*	Dereference	Used to dereference a pointer
->	Point-to	Used to access fields of a structure referenced by a pointer
+	Addition	Used to increment a pointer
-	Subtraction	Used to decrement a pointer
== !=	Equality, inequality	Compares two pointers
> >= < <=	Greater than, greater than or equal, less than, less than or equal	Compares two pointers
(data type)	Cast	To change the type of pointer

Pointer Arithmetic

Several arithmetic operations are performed on pointers to data. These include:

- Adding an integer to a pointer
- Subtracting an integer from a pointer
- Subtracting two pointers from each other
- Comparing pointers

These operations are not always permitted on pointers to functions

Adding an integer to a pointer

- This operation is very common and useful
- When we add an integer to a pointer, the amount added is the product of the integer times the number of bytes of the underlying data type
- The size of primitive data types can vary from system to system
- The common sizes found in most systems given below:

Data Type	Size in Bytes
byte	1
char	1
short	2
int	4
long	8
float	4
double	8

Adding an integer to a pointer

- Let's consider an array of integers, as shown below
- Each time one is added to pi, four is added to the address
- Pointers are declared with data types so that these sorts of arithmetic operations are possible

```c
#include <stdio.h>

int main(){

    int vector[] = {10,20,30,40,50};

    int *pi = vector;

    printf("%d \n", *pi); // Display 10

    pi += 1;
    printf("%d \n", *pi); // Display 20

    pi+= 1;
    printf("%d \n", *pi); // Display 30


}
```

When an array name is used by itself, it returns the address of an array,
which is also the address of the first element of the array

Subtracting an integer from a pointer

- Integer values can be subtracted from a pointer in the same way they are added
- The size of the data type times the integer increment value is subtracted from the address

```c
#include <stdio.h>

int main(){

    int vector[] = {10,20,30,40,50};

    int *pi = vector + 4;

    printf("%d \n", *pi); // Display 50

    pi--;
    printf("%d \n", *pi); // Display 40

    pi--;
    printf("%d \n", *pi); // Display 30

}
```

Subtracting two pointers

- When one pointer is subtracted from another, we get the difference between their addresses
- This difference is not normally very useful except for determining the order of elements in an array
- The difference between the pointers is the number of “units” by which they differ
- The difference’s sign depends on the order of the operands

```c
#include <stdio.h>

int main(){

    int vector[] = {10,20,30,40,50};

    int *p0 = vector;
    int *p1 = vector + 1;
    int *p2 = vector + 2;
    int *p3 = vector + 3;
    int *p4 = vector + 4;


    printf("p2-p0: %d\n",p2-p0); // p2-p0: 2
    printf("p2-p1: %d\n",p2-p1); // p2-p1: 1
    printf("p0-p1: %d\n",p0-p1); // p0-p1: -1

    return 0;
}
```

Comparing Pointers

- Pointers can be compared using the standard comparison operators
- Normally, comparing pointers is not very useful
- However, when comparing pointers to elements of an array, the comparison's results can be used to determine the relative ordering of the array's elements

```c
#include <stdio.h>

int main(){

    int vector[] = {10,20,30,40,50};

    int *p0 = vector;
    int *p1 = vector + 1;
    int *p2 = vector + 2;
    int *p3 = vector + 3;
    int *p4 = vector + 4;


    printf("p2>p0: %d\n",p2>p0); // p2>p0: 1
    printf("p2<p0: %d\n",p2<p0); // p2<p0: 0
    printf("p0>p1: %d\n",p0>p1); // p0>p1: 0

    return 0;
}
```

Multiple Levels of Indirection

- Pointers can use different levels of indirection
- It is not uncommon to see a variable declared as a pointer to a pointer, sometimes called a double pointer
- The example given in the next slide uses three arrays
- The first array is an array of strings used to hold a list of book titles
- Two additional arrays are provided whose purpose is to maintain a list of the “best books” and English books
- Instead of holding copies of the titles, they will hold the address of a title in the titles array
- Both arrays will need to be declared as a pointer to a pointer to a char
- The array’s elements will hold the addresses of the titles array’s elements
- This will avoid having to duplicate memory for each title and results in a single location for titles
- If a title needs to be changed, then the change will only have to be performed in one location

Multiple Levels of Indirection

```c
#include <stdio.h>

int main(){

    char *titles[] = {"A Tale of Two Cities",
        "Wuthering Heights","Don Quixote",
        "Odyssey","Moby-Dick","Hamlet",
        "Gulliver's Travels"
    };

    char **bestBooks[3];
    char **englishBooks[4];

    bestBooks[0] = &titles[0];
    bestBooks[1] = &titles[3];
    bestBooks[2] = &titles[5];

    englishBooks[0] = &titles[0];
    englishBooks[1] = &titles[1];
    englishBooks[2] = &titles[5];
    englishBooks[3] = &titles[6];

    printf("%s\n",*englishBooks[1]); // Wuthering Heights

    return 0;
}
```

Constants and Pointers

Using the **const** keyword with pointers is a rich and powerful aspect of C

- **Pointers to a constant**
  - A pointer can be defined to point to a constant
  - This means the pointer cannot be used to modify the value it is referencing
  - In the following example, an integer and an integer constant are declared. Next, a pointer to an integer and a pointer to an integer constant are declared and then initialized to the respective integers:

```c
#include <stdio.h>

int main(){

    int num = 5;
    const int limit = 500;
    int *pi; // pointer to an integer
    const int *pci; // pointer to a constant integer

    pi = &num;
    pci = &limit;

    printf("num   - Address : %p    value : %d\n",&num, num);
    printf("limit - Address : %p    value : %d\n",&limit, limit);
    printf("pi    - Address : %p    value : %p\n",&pi, pi);
    printf("pci   - Address : %p    value : %p\n",&pci, pci);

    return 0;
}
```

```
num   - Address : 00000000005FFECC    value : 5
limit - Address : 00000000005FFEC8    value : 500
pi    - Address : 00000000005FFEC0    value : 00000000005FFECC
pci   - Address : 00000000005FFEB8    value : 00000000005FFEC8
```

Pointers to a constant

- Dereferencing a constant pointer is fine if we are simply reading the value

```
printf("%d\n", *pci);
```

- We cannot dereference a constant pointer to change what the pointer references, but we can change the pointer as the pointer value is not constant
- The pointer can be changed to reference another constant integer or a simple integer

```
#include <stdio.h>

int main(){

    int num = 5;
    const int limit = 500;
    int *pi; // pointer to an integer
    const int *pci; // pointer to a constant integer

    pi = &num;
    pci = &limit;

    pci = &num; // valid assignment

    *pci = 200; // invalid, produce syntax error

    return 0;
}
```

Constant pointers to constants

- A constant pointer to a constant is an infrequently used pointer type
- The pointer cannot be changed, and the data it points to cannot be changed through the pointer
- An example of a constant pointer to a constant integer follows:

```
#include <stdio.h>

int main(){

    int num = 5;
    const int limit = 500;
    const int * const cpci = &limit;

    cpci = &num; // invalid, gives a syntax error

    return 0;
}
```

Constants and Pointers

Pointer Type	Pointer Modifiable	Data Pointed to Modifiable
Pointer to a nonconstant	✓	✓
Pointer to a constant	✓	X
Constant pointer to a nonconstant	X	✓
Constant pointer to a constant	X	X

Dynamic Memory Management in C

Dynamic Memory Management

- Much of the power of pointers stems from their ability to track dynamically allocated memory
- The management of this memory through pointers forms the basis for many operations, including those used to manipulate complex data structures
- The ability to allocate and then deallocate memory allows an application to manage its memory more efficiently and with greater flexibility
- Instead of having to allocate memory to accommodate the largest possible size for a data structure, only the actual amount required needs to be allocated

Languages such as C also support dynamic memory management where objects are allocated memory from the heap. This is done manually using functions to allocate and deallocate memory. The process is referred to as **dynamic memory management**.

Dynamic Memory Allocation

- The basic steps used for dynamic memory allocation in C are:
  - Use a **malloc** type function to allocate memory
  - Use this memory to support the application
  - Deallocate the memory using the **free** function

- While there are some minor variations to this approach, this is the most common technique

- In the next example, we allocate memory for an integer using the malloc function. The pointer assigns five to the allocated memory, and then the memory is released using the free function:

Dynamic Memory Allocation

```c
#include <stdio.h>   // Include standard input-output header for printf
#include <stdlib.h>  // Include standard library header for malloc and free

int main(){

    // Allocate memory dynamically for one integer
    int *pi = (int*)malloc(sizeof(int));

    // Assign value 5 to the allocated integer
    *pi = 5;

    // Print the value stored at the allocated memory
    printf("*pi: %d\n", *pi);

    // Free the allocated memory to avoid memory leak
    free(pi);

    // Return 0 to indicate successful program execution
    return 0;
}
```

Each time the malloc function (or similar function) is called, a corresponding call to the **free** function must be made when the application is done with the memory to avoid memory leaks.

Memory Leaks

- A memory leak occurs when allocated memory is never used again but is not freed
- This can happen when:
  - The memory's address is lost
  - The free function is never invoked though it should be (sometimes called a hidden leak)
- A problem with memory leaks is that the memory cannot be reclaimed and used later
- The amount of memory available to the heap manager is decreased

Lost Memory

- If memory is repeatedly allocated and then lost, then the program may terminate when more memory is needed but malloc cannot allocate it because it ran out of memory
- In extreme cases, the operating system may crash
- This is illustrated in the following simple example:

```
char *chunk;
while (1) {
    chunk = (char*) malloc(1000000);
    printf("Allocating\n");
}
```

The variable chunk is assigned memory from the heap. However, this memory is not freed before another block of memory is assigned to it. Eventually, the application will run out of memory and terminate abnormally. At minimum, memory is not being used efficiently.

Memory Leaks : Losing an Address

- An example of losing the address of memory is illustrated in the following code sequence where pi is reassigned a new address
- The address of the first allocation of memory is lost when pi is allocated memory a second time

```
int *pi = (int*) malloc(sizeof(int));
*pi = 5;
// ......
pi = (int*) malloc(sizeof(int));
```

Memory Leaks : Hidden Memory Leaks

- Hidden memory leaks occurs when the program should release memory but does not
- A hidden memory leak occurs when an object is kept in the heap even though the object is no longer needed
- This is frequently the result of programmer oversight
- The primary problem with this type of leak is that the object is using memory that is no longer needed and should be returned to the heap

```c
#include <stdio.h>   // Include standard input-output header for printf
#include <stdlib.h>  // Include standard library header for malloc and free

int main(){

    int *p = (int*)malloc(sizeof(int));  // allocate memory
    *p = 99;                             // use it

    // Forgot to call free(p)!
    // Hidden leak: memory was used, but never released.

    return 0;  // Leak persists until program exits

}
```

Dynamic Memory Allocation Functions

- Several memory allocation functions are available to manage dynamic memory
- While what is available may be system dependent, the following functions are found on most systems in the **stdlib.h** header file:
  - `malloc`
  - `realloc`
  - `calloc`
  - `free`

Function	Description
`malloc`	Allocates memory from the heap
`realloc`	Reallocates memory to a larger or smaller amount based on a previously allocated block of memory
`calloc`	Allocates and zeros out memory from the heap
`free`	Returns a block of memory to the heap

Using the malloc Function

- The function **malloc** allocates a block of memory from the heap
- The number of bytes allocated is specified by its single argument
- Its return type is a pointer to void
- If memory is not available, NULL is returned
- The function does not clear or otherwise modify the memory, thus the contents of memory should be treated as if it contained garbage

- The function's prototype follows:

```
void* malloc(size_t);
```

Using the malloc Function

- The following shows a typical use of the malloc function:

```
int *pi = (int*)malloc(sizeof(int));
```

- The following steps are performed when the malloc function is executed:
  1. Memory is allocated from the heap
  2. The memory is not modified or otherwise cleared
  3. The first byte's address is returned

Using the calloc Function

- The calloc function will allocate and clear memory at the same time
- To clear memory means its contents are set to all binary zeros
- Its prototype is as follows:

```
void *calloc(size_t numElements, size_t elementSize);
```

- The function will allocate memory determined by the product of the numElements and elementSize parameters
- A pointer is returned to the first byte of memory
- If the function is unable to allocate memory, NULL is returned
- If either numElements or elementSize is zero, then a null pointer may be returned

Using the calloc Function

- Consider the following example where pi is allocated a total of 20 bytes, all containing zeros:

```
int *pi = (int *)calloc(5,sizeof(int));
```

- Use calloc when memory needs to be zeroed out
- However, the execution of calloc may take longer than using malloc

Using the realloc Function

- Periodically, it may be necessary to increase or decrease the amount of memory allocated to a pointer
- This is particularly useful when a variable size array is needed
- The **realloc** function will reallocate memory
- Its prototype is as follows:

```
void *realloc(void *ptr, size_t size);
```

- The function realloc returns a pointer to a block of memory
- The function takes two arguments
- The first is a pointer to the original block, and the second is the requested size
- The reallocated block's size will be different from the size of the block referenced by the first argument
- The return value is a pointer to the reallocated memory

Using the realloc Function

- The requested size may be smaller or larger than the currently allocated amount
- If the size is less than what is currently allocated, then the excess memory is returned to the heap
- There is no guarantee that the excess memory will be cleared
- If the size is greater than what is currently allocated, then if possible, the memory will be allocated from the region immediately following the current allocation
- Otherwise, memory is allocated from a different region of the heap and the old memory is copied to the new region
- If the size is zero and the pointer is not null, then the pointer will be freed
- If space cannot be allocated, then the original block of memory is retained and is not changed

Behavior of realloc function

First Parameter	Second Parameter	Behavior
null	NA	Same as `malloc`
Not null	0	Original block is freed
Not null	Less than the original block's size	A smaller block is allocated using the current block
Not null	Larger than the original block's size	A larger block is allocated either from the current location or another region of the heap

realloc Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>


int main(){
    char *string1;
    char *string2;
    string1 = (char*) malloc(16);
    strcpy(string1,"0123456789AB");

    string2 = realloc(string1,8);

    printf("String 1 value: %p [%s]\n",string1, string1);
    printf("String 2 value: %p [%s]\n",string2, string2);

    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>


int main(){
    char *string1;
    char *string2;
    string1 = (char*) malloc(16);
    strcpy(string1,"0123456789AB");

    string2 = realloc(string1,80);

    printf("String 1 value: %p [%s]\n",string1, string1);
    printf("String 2 value: %p [%s]\n",string2, string2);

    return 0;
}
```

```
String 1 value: 000000000073A490 [01234567]
String 2 value: 000000000073A490 [01234567]
```

```
String 1 value: 000000000067A490 [g]
String 2 value: 000000000067E920 [0123456789AB]
```

Deallocating Memory Using the free Function

- With dynamic memory allocation, the programmer is able to return memory when it is no longer being used, thus freeing it up for other uses
- This is normally performed using the **free** function, whose prototype is shown below:

```
void free(void *ptr);
```

- The pointer argument should contain the address of memory allocated by a malloc type function
- This memory is returned to the heap
- While the pointer may still point to the region, always assume it points to garbage
- This region may be reallocated later and populated with different data
- In the simple example below, pi is allocated memory and is eventually freed:

Deallocating Memory Using the free Function

- In the simple example below, pi is allocated memory and is eventually freed:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>


int main(){
    int *pi = (int *)malloc(sizeof(int));
    // ............
    free(pi);

    return 0;
}
```

- If the free function is passed a null pointer, then it normally does nothing

Manage memory allocation/deallocation at the same level. For example,if a pointer is allocated within a function, deallocate it in the same function.

Assigning NULL to a Freed Pointer

- Pointers can cause problems even after they have been freed
- If we try to dereference a freed pointer, its behavior is undefined
- As a result, some programmers will explicitly assign **NULL** to a pointer to designate the pointer as invalid
- Subsequent use of such a pointer will result in a runtime exception
- An example of this approach is as follows:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>


int main(){
    int *pi = malloc(sizeof(int));
    // ............
    free(pi);
    pi = NULL;

    return 0;
}
```

Dangling Pointers

- If a pointer still references the original memory after it has been freed, it is called a **dangling pointer**
- The pointer does not point to a valid object
- This is sometimes referred to as a premature free
- The use of dangling pointers can result in a number of different types of problems, including:
  - Unpredictable behavior if the memory is accessed
  - Segmentation faults when the memory is no longer accessible
  - Potential security risks

- These types of problems can result when:
  - Memory is accessed after it has been freed
  - A pointer is returned to an automatic variable in a previous function call

Example : Dangling Pointers

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>


int main(){
    int *pi1 = malloc(sizeof(int));   // allocate memory for one int
    *pi1 = 50;                        // store 50 in allocated memory

    // ............
    free(pi1);                        // free the allocated memory
                                      // now pi1 becomes a dangling pointer
                                      // (it still points to the freed memory)

    // ............
    int *pi2 = pi1;                   // pi2 is assigned the same dangling address as pi1
                                      // both pi1 and pi2 are now invalid

    *pi2 = 200;                       // ❌ undefined behavior: writing to freed memory
    printf("%d\n",*pi1);              // ❌ undefined behavior: reading from freed memory
    printf("%d\n",*pi2);              // ❌ undefined behavior: reading from freed memory

    return 0;
}
```

Dealing with Dangling Pointers

- Debugging pointer-induced problems can be difficult to resolve at times. Several approaches exist for dealing with dangling pointers, including:
  - Setting a pointer to NULL after freeing it
  - Writing special functions to replace the free function
  - Some systems (runtime/debugger) will overwrite data when it is freed (e.g., 0xDEADBEEF - Visual Studio will use 0xCC, 0xCD, or 0xDD, depending on what is freed). While no exceptions are thrown, when the programmer sees memory containing these values where they are not expected, he knows that the program may be accessing freed memory
  - Use third-party tools to detect dangling pointers and other problems

Pointers and Functions

Example : Passing Data Using a Pointer

```c
#include <stdio.h>

void swapWithPointers(int *pnum1, int *pnum2){
    int temp;
    temp = *pnum1;
    *pnum1 = *pnum2;
    *pnum2 = temp;
}

int main(){
    int n1 = 5;
    int n2 = 10;
    swapWithPointers(&n1,&n2);

    printf("Value of n1: %d\n", n1); // outputs Value of n1: 10
    printf("Value of n2: %d\n",n2); // outputs Value of n2: 5
    return 0;
}
```

Activity : Explore how pass by reference and pass by value is different from each other

Returning a Pointer

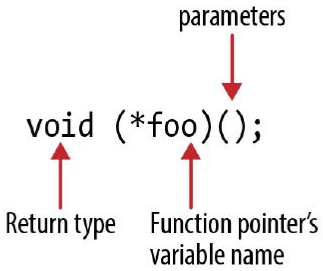
- If you want to return a pointer, you can simply declare the return type to be a pointer to the appropriate data type
- If we need to return an object from a function, the following two techniques are frequently used:
  - Allocate memory within the function using malloc and return its address. The caller is responsible for deallocating the memory returned
  - Pass an object to the function where it is modified. This makes the allocation and deallocation of the object's memory the caller's responsibility
- In the following example, we define a function that is passed the size of an integer array and a value to initialize each element. The function allocates memory for an integer array, initializes the array to the value passed, and then returns the array's address:

```
int* allocateArray(int size, int value){
    int* array = (int *)malloc(size * sizeof(int));

    for(int i = 0; i < size; i++){
        array[i] = value;
    }

    return array;
}
```

Function Pointers

- A function pointer is a pointer that holds the address of a function
- The ability of pointers to point to functions turns out to be an important and useful feature of C
- This provides us with another way of executing functions in an order that may not be known at compile time and without using conditional statements

- Declaring Function Pointers



Using a Function Pointer : Example

```c
#include <stdio.h>   // Standard input-output library for printf

// Declare a function pointer 'fptr' that points to a function
// taking an int as parameter and returning an int.
int (*fptr)(int);

// Define a function 'square' that returns the square of a number
int square(int number){
    return number * number;
}

int main(){

    int n = 5;  // Variable to hold the number to square

    // Assign the address of the function 'square' to the function pointer 'fptr'
    fptr = square;

    // Call the function 'square' indirectly using the function pointer 'fptr'
    // and print the result
    printf("%d square is %d\n", n, fptr(n));

    return 0;   // Exit successfully
}
```

Passing a Function Pointer : Example

```c
#include <stdio.h>

// Function to add two integers
int add(int num1, int num2){
    return num1 + num2;
}

// Function to subtract the second integer from the first
int substract(int num1, int num2){
    return num1 - num2;
}

// Define a type 'fptrOperation' as a pointer to a function
// that takes two integers as arguments and returns an integer.
typedef int (*fptrOperation)(int, int);
// 'compute' takes a function pointer (operation) and two integers.
// It calls the passed function pointer with the given numbers.
int compute(fptrOperation operation, int num1, int num2){
    return operation(num1, num2);
}

int main(){
    // Call 'compute' with the 'add' function and arguments (5,5).
    // Equivalent to add(5,5). Prints 10.
    printf("%d\n", compute(add, 5, 5));

    // Call 'compute' with the 'substract' function and arguments (15,5).
    // Equivalent to substract(15,5). Prints 10.
    printf("%d\n", compute(substract, 15, 5));

    return 0;   // Exit successfully
}
```

Any Questions?

- Post them in the discussion forum : Click here
- Send an email to : rns@ucsc.cmb.ac.lk